Project Documentation

LIBSPACE API – A ROBUST BACKEND FOR LIBRARY OPERATIONS

1. Introduction:

Project Title: Libspace API - A Robust Backend for Library Operations

Team ID: NM2025TMID36510

Team Size: 4

Team Leader: Mercy Cuety T 24csmercycuety056@gmail.com

Team Members:

-Monika M 24csmonikam057@gmail.com

-Sineha C 24cssinehac093@gmail.com

-Shabana Parveen M 24csshabana092@gmail.com

2. Project Overview:

Purpose:

- LibSpace is a micro web-based library management system.
- It provides secure user authentication and book management features.
- Users can register, login, logout and delete their accounts.
- After authentication users can perform CRUD operations on books.

Main Functions:

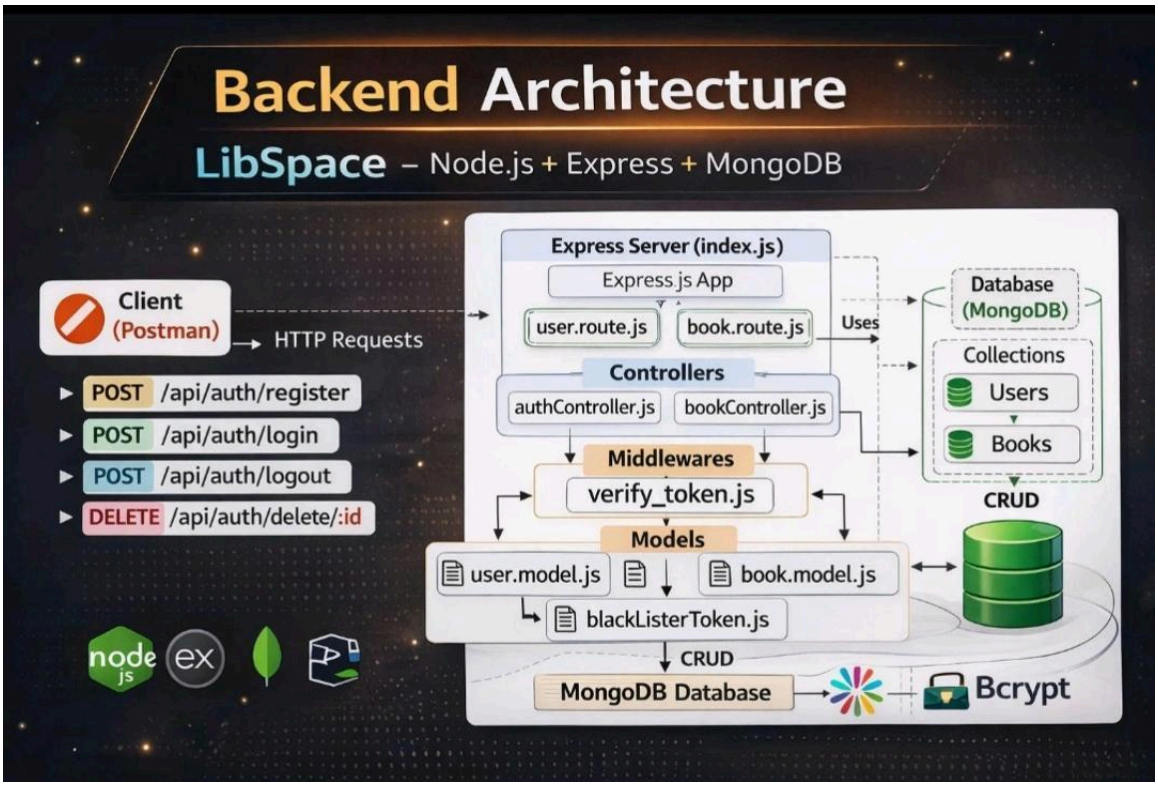
1. User registration and login.
2. Add new books.
3. View books list.
4. Update book details.
5. Delete books.

Key Features:

- User authentication system
- Book management system
- Secure API communication

- Database integration

3. Architecture:



Frontend:

- API testing using Postman

Backend:

- Node.js
- Express.js

Database:

- MongoDB

4. Setup Instructions:

Prerequisites:

- Node.js
- MongoDB
- Git
- Visual Studio Code

Installation Steps:

1. Clone the repository
   git clone <repo-link>

2. Install dependencies
   npm install

3. Run the application
   npm start

5. Folder Structure:

```
LIBSPACE/

│ -- config/
│    └── db.js

│ -- controllers/
│    └── authController.js
│    └── bookController.js

│ -- models/
│    └── user.model.js
│    └── book.model.js

│ -- routes/
│    └── user.route.js
│    └── book.route.js

│ -- middlewares/
│    └── verify_token.js

│ -- index.js
│ -- package.json
```

6. Running the Application:

Backend:

npm start

Access application:

http://localhost:5000

7. API Documentation:

User APIs:

- POST /api/auth/register – Register new user
- POST /api/auth/login – Login user
- DELETE /api/auth/delete – Delete user

Book APIs:

- POST /api/book/add – Add book
- GET /api/book – Fetch books
- PUT /api/book/:id – Update book
- DELETE /api/book/:id – Delete book

8. Authentication:

- JWT based authentication
- Middleware protects private routes
- Secure password encryption using bcrypt

9. User Interface:

- API responses are displayed in JSON format
- API testing performed using Postman

10. Testing:

Testing Tools:

- Postman
- Chrome Browser

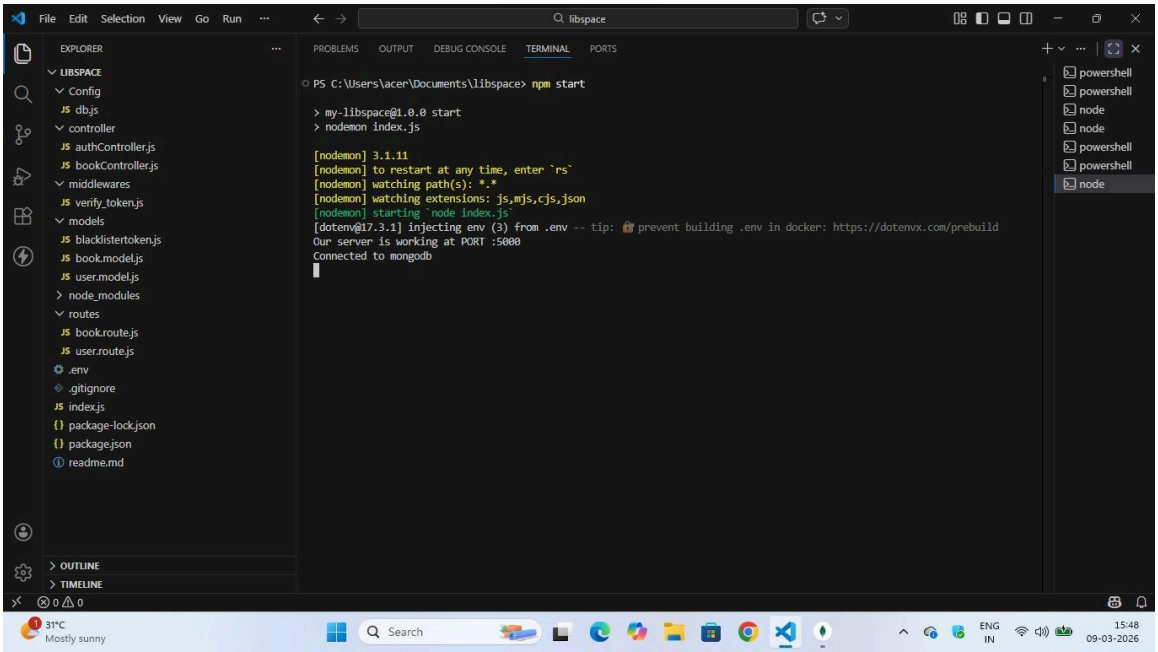
Testing includes:

- User registration
- User login
- Add book
- Get books
- Update book
- Delete book

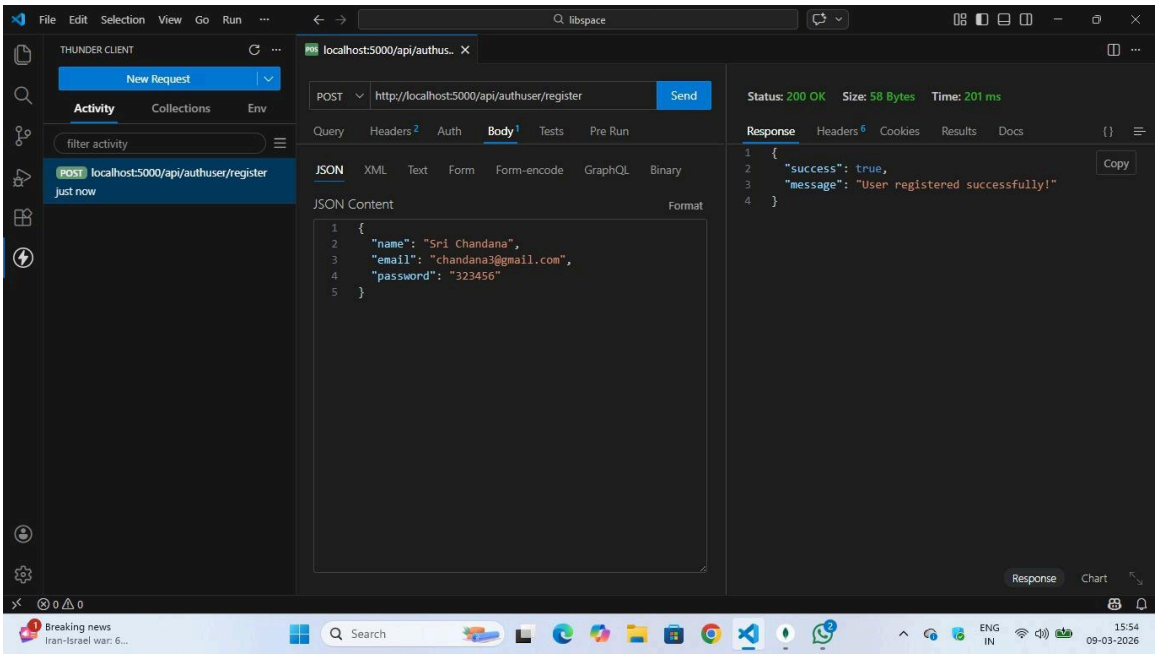
11. Screenshots / Demo:

Output Screenshots:

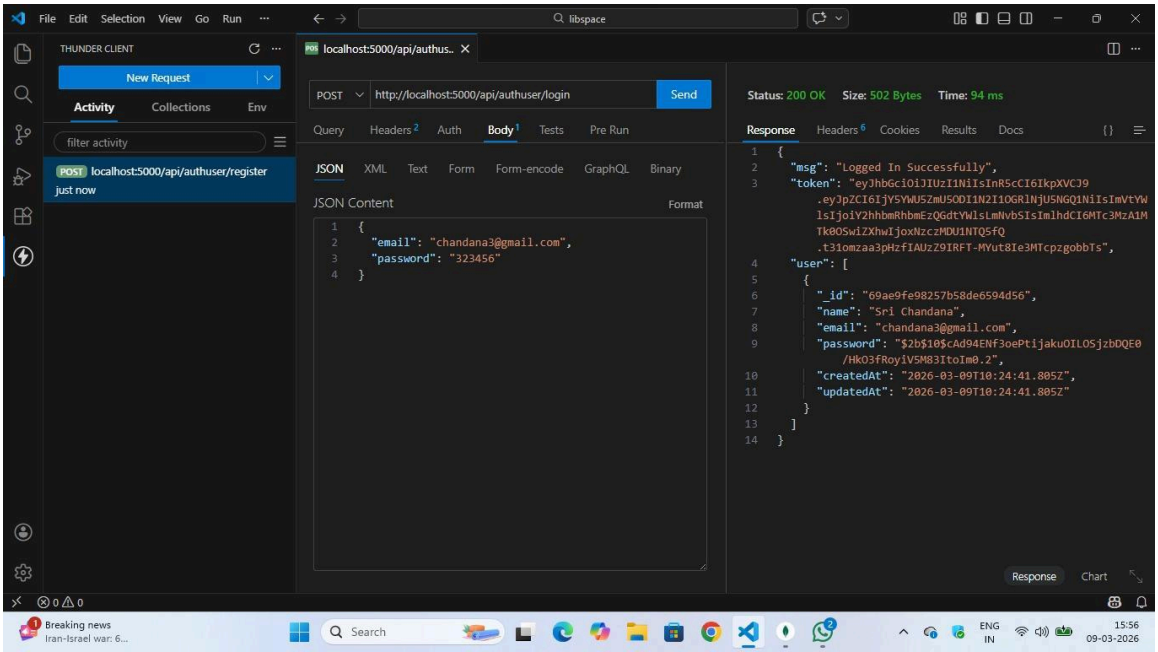
Output on terminal once server starts:



User registration output:



User Login output:



12. Known Issues:

- Database connection fails if MongoDB is not running.
- Token expires after certain time.
- Invalid inputs may return error responses.

13. Future Enhancements:

- Add frontend interface
- Implement search functionality
- Role-based authentication
- Mobile application support